

Set up the practicals with uv

For Windows, macOS and Linux. No existing Python installation is needed.

What you are setting up: uv installs Python and the course packages in a separate `.venv` folder. JupyterLab is the interface you open in your browser; a *kernel* is the Python process that runs notebook cells. Your R installation stays unchanged.

1 Download the supplied folder and open a terminal

Download the folder supplied by your instructor and extract the ZIP file. It may be the setup-only package or, later, the complete course repository. On Windows, right-click the ZIP and choose **Extract All**. On macOS, double-click it. Work in the extracted folder, not inside the ZIP.

The folder must contain `pyproject.toml`, `uv.lock`, `.python-version`, `scripts`, `setup_assets` and `JUPYTER_BASICS.ipynb`. These files are supplied: you do not need to write or generate them. A file beginning with a dot may be hidden in your file browser.

Windows: click Start or the Windows search bar, type **PowerShell**, and open Windows PowerShell. A terminal is simply a window where you type commands. Administrator mode is not needed for the course commands.

macOS: press **Command+Space**, type **Terminal**, and press Return. **Linux:** open the application named Terminal; many desktops also use Ctrl+Alt+T.

2 Install uv once

Type or paste each command into the terminal and press Enter. Do not include the terminal's prompt (such as `PS C:\...\>`). These commands are not R code and do not go in RStudio or a notebook cell.

Windows PowerShell:

```
winget install --id=astral-sh.uv -e
```

macOS / Linux Terminal:

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

After installation, close and reopen the terminal, then run:

```
uv --version
```

A version number confirms uv is available. If Windows says winget is not recognised, use the Windows standalone installer in the [official uv installation guide](#), then reopen PowerShell.

You need internet during the first setup. uv downloads Python automatically. Continue on the next page to locate your course folder. Reference: [uv with Jupyter](#). Windows setup is tested; macOS/Linux instructions still require an instructor pilot.

Install and open the course

Keep using the terminal window from page 1. Run one command at a time.

3 Tell the terminal which folder to use

The terminal has a **current folder**, which can differ from the folder shown in your file browser. `cd` means change directory (folder). Example: if Alex extracted the course into Downloads, the commands would be:

Windows PowerShell:

```
cd "C:\Users\Alex\Downloads\ESBMTK-practicals"
```

macOS / Linux:

```
cd "$HOME/Downloads/ESBMTK-practicals"
```

Use your actual folder name and location, including a suffix such as `-main` if present. On Windows, open the folder in File Explorer, click its address bar and copy that path between the quotes after `cd`.

Check your location with these commands (they work in PowerShell and macOS/Linux Terminal):

```
pwd
ls
```

`pwd` prints the current folder. In the `ls` listing, find **pyproject.toml**, **uv.lock**, **scripts**, **setup_assets** and **JUPYTER_BASICS.ipynb**. If they are missing, enter the correct folder before continuing. “From the supplied folder” always means this location in the terminal.

4 Install the packages and check them

```
uv sync --locked
uv run --locked python scripts/check_environment.py
```

The first command downloads Python and the locked packages into `.venv`; wait for it to finish. The second should end with **Environment check passed. Next: start JupyterLab**. It checks numerical calculations, chemistry, access to the supplied workbook probe and plotting. If it reports an error, use the help section on page 3.

5 Register the course kernel and start JupyterLab

Stay in this same terminal folder. Register the kernel once, then launch:

```
uv run --locked python -m ipykernel install --sys-prefix --name esbmtk-practicals
uv run --locked jupyter lab
```

JupyterLab opens in your browser. Keep the terminal open: the running server uses it, so the prompt will not return yet. If no browser opens, copy the local `http://localhost:.../lab` or `http://127.0.0.1:.../lab` link printed in the terminal into your browser. Keep the full link private.

Check success and start working

The steps on this page explain what to do in JupyterLab and when you are finished.

6 Complete the JupyterLab setup check

In JupyterLab's left file browser, double-click **JUPYTER_BASIC.ipynb**. Choose **esbmtk-practicals** if JupyterLab asks for a kernel. Follow the notebook from top to bottom, or select **Run > Run All Cells**. The last cell should print:

```
Jupyter setup check passed
```

The notebook also introduces the cell shortcuts used during the practicals. **If the terminal check and notebook check both pass, setup is complete.** You do not repeat installation or kernel registration each time.

Open or create notebooks while JupyterLab is running

Use the left file browser to double-click any notebook supplied by your instructor. If it asks for the instructor's ESBMTK314 kernel, choose **esbmtk-practicals** instead. Use **File > New > Notebook** to create another notebook. No terminal command is needed for each notebook. Save with Ctrl+S (Cmd+S on macOS).

When you finish: save and shut down

Save your notebooks first. Return to the terminal running JupyterLab, press **Ctrl+C**, and, if asked to shut down, type **y** and press Enter. Wait for the normal terminal prompt to return, then close the browser tab and terminal. This stops notebook kernels, so unsaved variables are lost; saved notebooks remain on disk.

Closing the browser tab alone does not normally stop JupyterLab. If you only closed the tab, reopen the local link still shown in its terminal; you do not need to start another server.

Next time: open JupyterLab again

Open PowerShell or Terminal, repeat `cd` to the supplied folder (page 2), and run:

```
uv run --locked jupyter lab
```

This is only for when the server has stopped, including after restarting your laptop. Open your saved notebooks and rerun the cells needed to recreate variables.

If a step fails

uv not recognised: reopen the terminal. **Project/file not found:** check `pwd` and `ls` as on page 2. **Missing or outdated lockfile:** obtain the complete current supplied folder; keep `--locked`. **Wrong Python path or missing package in a notebook:** select the course kernel and restart that kernel. For other errors, send the full error text, the command you ran and your operating system to your instructor.